UNIVERSIDAD AUTONOMA DE CHILE

Introducción a la Robótica

Diseña, imprime y programa tu propio robot futbolista

Guía paso a paso: diseño CAD en OnShape, impresión 3D en PLA, electrónica con NodeMCU ESP8266 y módulo L298N, programación en C++ y control por Bluetooth desde el celular.

Eduardo Fuentes Fierro
Ingeniero Civil Mecánico
Magíster en Ciencias de la Ingeniería

Primera edición · 2026

Introducción a la Robótica
Diseña, imprime y programa tu propio robot futbolista

Primera edición, 2026.

Eduardo Fuentes Fierro
Ingeniero Civil Mecánico
Magíster en Ciencias de la Ingeniería
Universidad Autónoma de Chile

Esta obra se distribuye bajo licencia
Creative Commons Atribución-NoComercial-CompartirIgual 4.0 Internacional (CC BY-NC-SA 4.0)

Eres libre de copiar, distribuir y adaptar este material citando la fuente, sin fines comerciales y compartiendo las obras derivadas bajo la misma licencia.

Los archivos del proyecto (código para el NodeMCU, app de App Inventor, modelos STL y este libro en PDF) están disponibles para descarga libre en el repositorio del libro:
`https://github.com/Edward3th/UA_Robotica`

Compuesto con LaTeX (LuaLaTeX, clase KOMA-Script).
Tipografías: TeX Gyre Pagella, TeX Gyre Heros y TeX Gyre Adventor.

El logotipo de la Universidad Autónoma de Chile se reproduce con fines identificatorios. Los nombres de productos y marcas mencionados (Arduino, NodeMCU, Espressif ESP8266, OnShape, Ultimaker Cura, MIT App Inventor) pertenecen a sus respectivos titulares y se citan con fines educativos.

Índice general

0 Prólogo: ¿por qué construir un robot? **1**
0.1 ¿Qué es la robótica? 1
0.2 La robótica hoy 1
0.3 Qué construirás en este libro 2
0.4 Cómo usar este libro 3

1 Materiales y software **5**
1.1 Lista de materiales 5
1.2 Software que vas a instalar 7

2 Diseño CAD en OnShape **9**
2.1 ¿Qué es el CAD paramétrico? 9
2.2 La interfaz de OnShape 9
2.3 Croquis y restricciones 9
2.4 Operaciones básicas 10
2.5 Manos a la obra: diseña tu chasis 11
2.6 Exportar a STL 11
2.7 El chasis del autor 12

3 Impresión 3D **15**
3.1 Manufactura aditiva: qué es y por qué FDM 15
3.2 El material: PLA 15
3.3 Diseño para impresión 3D 16
3.4 Laminando en Cura 16
3.5 Manos a la obra: imprime tu chasis 17
3.6 Fallas típicas 17

4 Electrónica **19**
4.1 Conceptos base 19
4.2 Descripción de componentes 19
4.3 Por qué necesitas un puente H 19
4.4 Servomotores y ancho de pulso 21
4.5 Alimentación 21
4.6 Manos a la obra: cableado y ensamble 22

5 Programación **27**
5.1 ¿Qué es programar? 27
5.2 Prepara el Arduino IDE para el ESP8266 27
5.3 Estructura de un programa 28
5.4 De un motor a dos: funciones y tracción diferencial 29
5.5 Servos y la función de pateo 30
5.6 Manos a la obra: tu rutina de prueba 31
5.7 Depuración con el Monitor Serie 31

6 Comunicación **33**
6.1 ¿Qué es un protocolo, y por qué Bluetooth? 33
6.2 El módulo HC-05 y el puerto serial 33

6.3 El protocolo de comandos del robot 34
6.4 La app de control en MIT App Inventor 36
6.5 Manos a la obra: empareja y controla tu robot 37

7 Epílogo 39
7.1 Reglas para jugar 39
7.2 Lo que aprendiste 39
7.3 Hacia dónde seguir 39

A Diagrama de conexiones 41

B Guía de resolución de problemas 43

C Archivos descargables 45

0 Prólogo: ¿por qué construir un robot?

EN ESTE CAPÍTULO APRENDERÁS

- Entender qué es la robótica y por qué es un campo interdisciplinario.
- Reconocer robots que ya te rodean y situar el fútbol robótico como problema de aprendizaje.
- Saber qué construirás exactamente a lo largo de este libro.
- Conocer cómo está organizado el libro y cómo sacarle provecho.

0.1 ¿Qué es la robótica?

Un robot no es una sola disciplina: es el punto donde varias se encuentran. La **mecánica** le da un cuerpo capaz de moverse (ruedas, ejes, engranajes). La **electrónica** le entrega lógica y control (motores, servos, sensores). La parte **eléctrica** resuelve cómo llega la energía desde la batería hasta cada componente sin que nada se queme en el camino. Y la **informática**, a través de un programa, decide *qué* hace el robot y *cuándo* lo hace.

Ningún robot funciona si falla una sola de estas cuatro capas. Un chasis perfecto con una batería mal conectada no se mueve. Un motor bien alimentado sin código no sabe qué hacer. Por eso la robótica se aprende mejor construyendo: cada capítulo de este libro resuelve una capa a la vez, y solo al final las cuatro conviven en la misma máquina.

NOTA

No necesitas saber nada de electrónica, CAD ni programación para empezar. Esta es una guía práctica que asume que no tienes experiencia previa.

0.2 La robótica hoy

Ya convives con robots todos los días, aunque no siempre los llames así: el brazo que suelda carrocerías en una fábrica de autos, el robot aspirador que esquiva los muebles de una casa, el dron que entrega un paquete. Todos comparten la misma estructura de fondo que vas a construir en este libro: sensores o entradas que informan del entorno, un controlador que decide, y actuadores que ejecutan esa decisión sobre el mundo físico. La Figura 0.1 muestra un ejemplo de cada contexto: la industria, el hogar y la competencia.

Para aprender robótica hace falta un problema concreto, no solo teoría. La **RoboCup** —la competencia internacional de fútbol robótico— es un buen problema porque obliga a resolver movimiento, control y comunicación a la vez, con una meta simple de entender: mover un balón hacia un arco. No vas a competir en la RoboCup, pero vas a construir el mismo tipo de máquina que ahí compite, a una escala pensada para tu primer robot.



Figura 0.1. Robots presentes en distintos contextos: industria, hogar y competencia. Fotografías: Phasmatisnox (CC BY 3.0), Air fans (CC0) y Eneas De Troya (CC BY 2.0); montaje y recortes propios.

0.3 Qué construirás en este libro

A lo largo de estos capítulos vas a diseñar, imprimir, cablear, programar y controlar un robot futbolista: un vehículo de **tracción diferencial** —dos motores DC independientes, uno por lado— con dos servomotores que actúan como paletas laterales, al estilo de un pinball, para golpear un balón. Su cerebro es un NodeMCU, una placa pequeña basada en el chip ESP8266, y lo vas a controlar de forma inalámbrica, por Bluetooth, desde una aplicación de celular hecha en MIT App Inventor que vas a instalar, entender por dentro y adaptar. La Figura 0.2 muestra el resultado.

[IMAGEN PENDIENTE]

`cap00-02-robot-terminado`

Figura 0.2. El robot futbolista terminado: vista general con sus componentes principales.

CONSEJO PRÁCTICO

Si en algún capítulo no ves de inmediato para qué sirve lo que estás construyendo, vuelve a la Figura 0.2. Cada pieza del libro —cada motor, cada línea de código, cada conexión

Bluetooth— existe para que este robot termine funcionando.

0.4 Cómo usar este libro

El libro está organizado como una secuencia de construcción, no como un manual de referencia: cada capítulo se apoya en el anterior y te acerca un paso más al robot terminado. Vas a encontrar siete elementos recurrentes que marcan el ritmo de la lectura:

- Una caja de **objetivos** al abrir cada capítulo, como la de arriba.
- Cajas de **nota** con información complementaria.
- Cajas de **atención** cuando hay riesgo real de daño a un componente.
- Cajas de **consejo práctico** con atajos y errores comunes.
- Cajas de **manos a la obra** con la tarea concreta que ejecutas.
- Cajas de **antes de seguir, verifica** al cierre de cada sección clave.
- Un **resumen** al final de cada capítulo.

No necesitas ningún conocimiento previo de electrónica, CAD ni programación, ni tampoco herramientas más allá de las que se listan en el Capítulo 1. Lo que sí necesitas es continuar los capítulos en orden: cada uno da por hecho lo que construiste en el anterior.

RESUMEN DEL CAPÍTULO

- La robótica combina mecánica, eléctrica, electrónica e informática: ninguna capa funciona sola.
- Los robots que ya te rodean comparten la misma estructura —sensores, control, actuadores— que vas a construir en este libro.
- Vas a construir un robot futbolista de tracción diferencial, con paletas tipo pinball, controlado por Bluetooth desde tu celular.
- El libro se lee en orden: cada capítulo depende de lo construido en el anterior.

1 Materiales y software

EN ESTE CAPÍTULO APRENDERÁS

- Reunir todos los componentes y herramientas que vas a necesitar.
- Instalar y probar los cuatro programas que vas a usar en este libro.
- Llegar al Capítulo 2 con todo listo, sin interrupciones a mitad de camino.

Antes de tocar cualquier herramienta conviene reunir todo el material. Interrumpir el Capítulo 4 a mitad del armado porque falta un componente es la forma más común de perder el impulso de un proyecto. Este capítulo cierra ese riesgo.

1.1 Lista de materiales

La Tabla 1.1 reúne todo lo que necesitas para construir el robot completo. Ningún componente se usa antes del Capítulo 4, así que puedes seguir comprando o esperando envíos mientras avanzas con los Capítulos 2 y 3.

ATENCIÓN

El shield `HW-588A` solo calza con el NodeMCU angosto (versión «Amica», con chip `CP2102`). El NodeMCU «LoLin V3» es más ancho y no entra en sus conectores. Revisa la foto y las medidas antes de comprar.

NOTA

Muchos cables micro-USB baratos solo sirven para cargar: no llevan las líneas de datos y la computadora nunca detecta la placa. Si en el Capítulo 5 el NodeMCU no aparece, prueba primero con otro cable.

CONSEJO PRÁCTICO

Además de los componentes, necesitas herramientas básicas: un destornillador pequeño, un alicate de corte, y —muy recomendable aunque no obligatorio— un multímetro para verificar voltajes antes de conectar cada etapa nueva. Un soldador es opcional: todo el cableado de este libro se resuelve con borneras de tornillo y cables jumper, sin soldadura.

NOTA

No necesitas tener impresora 3D propia. El Capítulo 3 asume que vas a usar una impresora de acceso compartido (de tu universidad, un makerspace o un servicio de impresión); lo único tuyo es el filamento y el diseño.

Tabla 1.1. Materiales necesarios para el robot futbolista.

Componente	Cant.	Rol en el robot
NodeMCU ESP8266 v1.0 (ESP-12E, chip USB `CP2102`)	1	Microcontrolador, cerebro del sistema
Shield `HW-588A` para NodeMCU	1	Base del NodeMCU: reparte la alimentación y trae pines para conectar todo sin soldar
Módulo puente H `L298N`	1	Control de potencia y sentido de los motores DC
Motor DC con reductora + rueda	2	Tracción diferencial (avance, retroceso, giro)
Servomotor (SG90 o MG90S)	2	Paletas laterales tipo pinball, para golpear el balón
Módulo Bluetooth `HC-05`	1	Enlace inalámbrico con la app del celular
Rueda loca / caster	1	Tercer punto de apoyo del chasis
Portapilas 4×AA con interruptor	1	Alimentación (6 V)
Pilas AA alcalinas	4	Energía para todo el robot
Capacitor electrolítico $\geq$470 µF, $\geq$16 V	1	Desacople del bus de alimentación cerca de los servos
Cables jumper hembra-hembra	~12	Señales entre el shield, el `L298N` y el `HC-05`
Cable rojo y negro para borneras	1 m	Alimentación entre pilas, `L298N` y shield
Cable USB a micro-USB, con datos	1	Programar el NodeMCU desde tu computadora
Celular Android	1	Corre la app de control (Capítulo 6)
Filamento PLA	1 rollo	Impresión del chasis (Capítulo 3)
Pelota de tenis	1	Balón del robot

1.2 Software que vas a instalar

Vas a usar cuatro programas a lo largo del libro. Ninguno es gratuito por casualidad: los cuatro tienen una versión gratuita o educativa suficiente para este proyecto.

OnShape
: CAD paramétrico que corre en el navegador, sin instalación. Lo usas en el Capítulo 2 para diseñar el chasis del robot.

Ultimaker Cura
: Convierte tu diseño 3D en instrucciones que entiende la impresora (*slicing*). Lo usas en el Capítulo 3.

Arduino IDE
: Entorno donde escribes y cargas el código al NodeMCU. Aunque la placa no es un Arduino, se programa igual: basta agregarle al IDE el paquete del ESP8266 (Capítulo 5). Lo usas desde el Capítulo 5 en adelante.

MIT App Inventor
: Herramienta web para construir aplicaciones de celular. En el Capítulo 6 abres en ella la app que controla el robot.

La Figura 1.1 muestra los cuatro programas juntos.



Figura 1.1. Los cuatro programas que usarás en este libro: OnShape, Ultimaker Cura, Arduino IDE y MIT App Inventor.

MANOS A LA OBRA

Prepara tu equipo antes de continuar al Capítulo 2:

1. Crea una cuenta gratuita en `onshape.com` (solo necesitas correo).
2. Descarga e instala Ultimaker Cura desde su sitio oficial.
3. Descarga e instala el Arduino IDE desde `arduino.cc`. El paquete del ESP8266 lo agregas en el Capítulo 5.
4. Crea una cuenta gratuita en `appinventor.mit.edu` (con cuenta Google).
5. Abre los cuatro programas al menos una vez y confirma que inician sin errores.

ANTES DE SEGUIR, VERIFICA

- [ ] Tienes en mano (o en camino) todos los componentes de la Tabla 1.1.
- [ ] Tu NodeMCU es la versión angosta, compatible con el shield `HW-588A`.
- [ ] Tu cuenta de OnShape está creada y abre el editor.
- [ ] Cura y el Arduino IDE están instalados y abren sin errores.
- [ ] Tu cuenta de MIT App Inventor está creada.

RESUMEN DEL CAPÍTULO

- El robot necesita un microcontrolador (NodeMCU ESP8266) montado en su shield, dos motores DC, dos servos, un puente H, un módulo Bluetooth y una fuente de alimentación: nada de eso se usa antes del Capítulo 4, pero conviene tenerlo listo desde ahora.
- Vas a usar cuatro programas, uno por etapa del proyecto: OnShape (diseño), Cura (impresión), Arduino IDE (programación) y MIT App Inventor (comunicación).
- No necesitas soldador ni impresora 3D propia para seguir este libro.

2 Diseño CAD en OnShape

EN ESTE CAPÍTULO APRENDERÁS

- Entender qué es el CAD paramétrico y por qué lo vas a usar.
- Moverte con soltura en la interfaz de OnShape.
- Crear un croquis con restricciones y convertirlo en una pieza sólida.
- Diseñar el chasis de tu robot y exportarlo listo para imprimir.

2.1 ¿Qué es el CAD paramétrico?

CAD (diseño asistido por computadora) es la herramienta con la que un ingeniero dibuja una pieza antes de fabricarla. La palabra clave es **paramétrico**: cada medida de tu diseño queda guardada como un número que puedes cambiar después. Si dibujas un agujero de 5 mm y luego necesitas 6 mm, no rehaces el dibujo: cambias el número.

Vas a usar **OnShape** porque corre en el navegador —sin instalación ni licencia— y porque cualquier cambio se guarda solo, en la nube.

2.2 La interfaz de OnShape

Al abrir un documento nuevo vas a ver tres zonas: el árbol de *features* a la izquierda (el historial de todo lo que has hecho), el área de trabajo en el centro, y la barra de herramientas de croquis y operaciones arriba (Figura 2.1).

NOTA

Cada pestaña en la parte inferior es un *Part Studio* o un *Assembly*. En este capítulo trabajas siempre dentro de un único Part Studio: ahí vive el chasis completo como una sola pieza.

2.3 Croquis y restricciones

Todo sólido en OnShape empieza como un croquis plano (*sketch*) dibujado sobre una cara o un plano de referencia.

1. Selecciona el plano superior (*Top*) y entra en modo croquis (*Sketch*).
2. Dibuja un rectángulo con la herramienta *Corner rectangle*.
3. Acota el ancho y el alto con la herramienta *Dimension*: escribe el valor de cada cota.



Figura 2.1. Interfaz de OnShape: árbol de features (a), área de trabajo (b) y barra de herramientas (c).

4. Centra el rectángulo en el origen: dibuja una diagonal entre dos esquinas opuestas, conviértela en línea de construcción (*Construction*) y aplica la restricción *Midpoint* entre el origen y esa diagonal. Así el rectángulo queda centrado aunque cambies las medidas después.
5. Confirma que el croquis está completamente restringido: OnShape lo dibuja en negro. Si queda algo en azul, falta una restricción o una cota.

La Figura 2.2 muestra el croquis en tres momentos: recién dibujado, al aplicar la restricción *Midpoint* y totalmente restringido.



(**a**) Rectángulo inicial



(**b**) Restricción de punto medio



(**c**) Croquis totalmente restringido

Figura 2.2. Creación de un croquis base con restricciones.

CONSEJO PRÁCTICO

Si un croquis se ve azul y no sabes qué falta, arrastra con el mouse uno de los puntos azules: lo que se mueve es lo que aún no está restringido. Si algo se ve en rojo, hay restricciones en conflicto. Cuando el croquis se complica, el *Constraint Manager* (una herramienta de diagnóstico del croquis) lista las restricciones y te ayuda a encontrar la que falta o la que sobra. Aprender a leer los colores te va a servir en cada pieza del chasis.

2.4 Operaciones básicas

Con el croquis cerrado, tres operaciones te alcanzan para la mayoría de las piezas de este proyecto:

1. **Extrusión (Extrude):** convierte el croquis plano en un sólido con espesor. Es la operación que más vas a usar.
2. **Corte (Extrude con la opción Remove):** quita material siguiendo un croquis nuevo dibujado sobre una cara existente. Así se hacen los agujeros para tornillos y los alojamientos de los servos.
3. **Redondeo (Fillet):** suaviza una arista. Además de verse mejor, una arista redondeada imprime en 3D con menos problemas que una esquina viva.

La Figura 2.3 muestra las tres operaciones aplicadas sobre la misma placa.



(**a**) Extrusión de la base



(**b**) Corte para un alojamiento



(**c**) Redondeo de aristas

Figura 2.3. Las tres operaciones básicas aplicadas sobre la placa base.

2.5 Manos a la obra: diseña tu chasis

MANOS A LA OBRA

1. Mide físicamente cada componente de la Tabla 1.1 (Capítulo 1): NodeMCU montado en su shield `HW-588A`, módulo `L298N`, motores con reductora, servos, módulo `HC-05`, portapilas y rueda caster.
2. Decide dónde va cada componente sobre una placa base. Deja espacio para el cableado y para que el portapilas se pueda retirar sin desarmar nada. Mantén la placa dentro del volumen de impresión de tu impresora (en muchas es de unos 220×220 mm).
3. Dibuja un boceto a mano de la vista superior, con las medidas que acabas de tomar. No lo saltes: dibujar a mano obliga a decidir la distribución antes de pelear con el mouse.
4. Recién ahora, traslada ese boceto a un croquis en OnShape y extrúyelo en la placa base.
5. Agrega los cortes para fijar cada componente (ranuras para el módulo `L298N`, alojamientos para los servos, orificios para el shield del NodeMCU). Deja a la vista el conector USB del NodeMCU: vas a enchufarlo cada vez que cargues un programa.

ATENCIÓN

La boca frontal de tu chasis, por donde entra el balón, debe dejar pasar al menos un tercio del diámetro de una pelota de tenis (~67 mm). Una abertura más angosta impide que el balón llegue a las paletas, sin importar qué tan bien funcione el resto del robot. Verifica esta medida en tu croquis antes de extruir.

2.6 Exportar a STL

1. En el árbol de features, selecciona la pieza terminada.
2. Haz clic derecho y elige *Export*.
3. Elige formato `STL`, resolución *Fine*, y descarga el archivo.

NOTA

Guarda el archivo con un nombre claro, por ejemplo `chasis-robot.stl`. Lo vas a necesitar abierto en Cura en el Capítulo 3.

2.7 El chasis del autor

La Figura 2.4 muestra el chasis que se usó para escribir este libro. Tómalo como referencia de proporciones y ubicación de componentes, no como una plantilla obligatoria: tu propio diseño puede —y probablemente debería— verse distinto. Sus archivos STL están en el repositorio del libro (Anexo C).

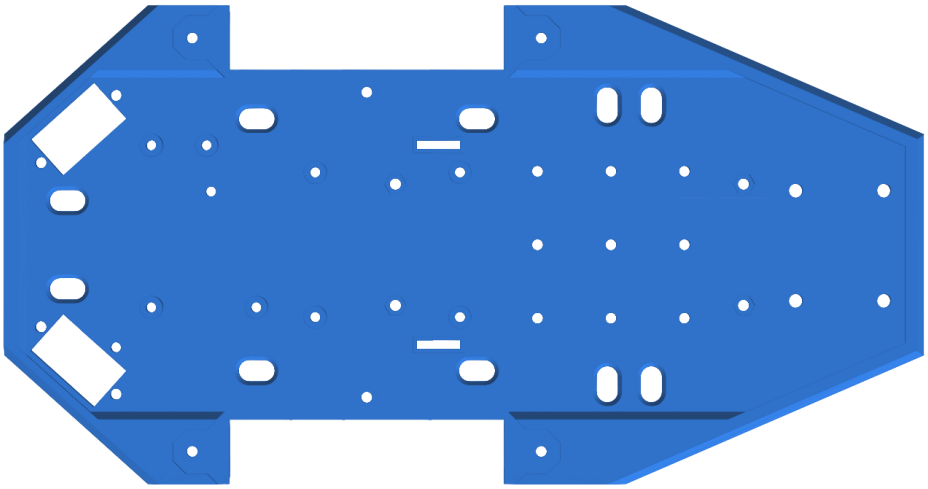


Figura 2.4. Placa del chasis de referencia del autor, vista desde arriba (250×130 mm).

ANTES DE SEGUIR, VERIFICA

- ☐ Tu croquis base está completamente restringido (en negro, sin azul).
- ☐ El chasis tiene espacio marcado para cada componente de la Tabla 1.1 que va montado en el robot.
- ☐ La boca frontal deja pasar al menos un tercio del diámetro de la pelota de tenis.
- ☐ Exportaste un archivo `.stl` y lo tienes guardado.

RESUMEN DEL CAPÍTULO

- El CAD paramétrico guarda cada medida como un número editable: por eso conviene restringir bien los croquis desde el principio.
- Extrusión, corte y redondeo alcanzan para diseñar la mayoría de las piezas de este

proyecto.
- El chasis se diseña a partir de mediciones reales de tus componentes, no al revés.
- La geometría de la boca frontal está condicionada por el tamaño del balón: es una restricción de diseño, no un detalle estético.

3 Impresión 3D

EN ESTE CAPÍTULO APRENDERÁS

- Entender qué es la manufactura aditiva y cómo imprime una impresora FDM.
- Conocer el PLA: sus temperaturas, ventajas y límites.
- Preparar tu chasis en Ultimaker Cura y convertirlo en instrucciones de impresión.
- Imprimir el chasis que diseñaste en el Capítulo 2.

3.1 Manufactura aditiva: qué es y por qué FDM

Fabricar una pieza por **manufactura sustractiva** significa partir de un bloque de material y quitarle lo que sobra —como tallar o fresar—. Fabricarla por **manufactura aditiva** significa lo contrario: construirla agregando material capa por capa, desde cero, hasta completar la forma.

La tecnología que vas a usar es **FDM** (Modelado por Deposición Fundida): un filamento sólido se funde en una boquilla caliente y se deposita capa sobre capa, siguiendo exactamente el camino que definiste al diseñar el chasis en OnShape. En la Figura 3.1 se ven las tres partes que importan: la boquilla, la cama caliente y los ejes de movimiento.

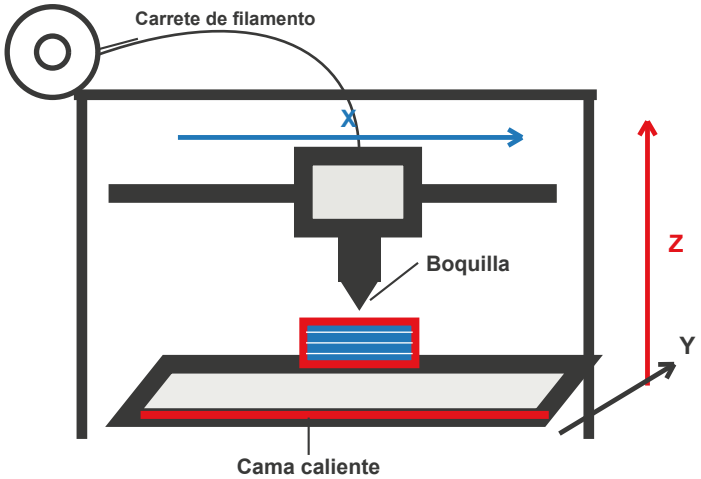


Figura 3.1. Anatomía de una impresora FDM: boquilla, cama caliente y ejes de movimiento.

3.2 El material: PLA

Vas a imprimir en **PLA** (ácido poliláctico), el material más usado para empezar en impresión 3D: es fácil de imprimir, no necesita cama caliente a temperaturas extremas y no desprende gases fuertes.

Tabla 3.1. PLA comparado con otros filamentos comunes.

Material	Boquilla	Cama	Para este proyecto
PLA	190–220 °C	50–60 °C	Recomendado: fácil, rígido, suficiente para el chasis
PETG	230–250 °C	70–80 °C	Más resistente al impacto, más difícil de imprimir bien
ABS	220–250 °C	90–110 °C	Requiere recinto cerrado; se deforma (*warping*) sin él

NOTA

El PLA es más frágil que el PETG o el ABS a temperatura ambiente, pero para un chasis que no queda expuesto al sol ni a temperaturas altas, esa diferencia no importa. Empieza con PLA; puedes reimprimir en otro material más adelante si una pieza en particular lo necesita.

3.3 Diseño para impresión 3D

Antes de laminar el chasis del Capítulo 2, revisa estos cuatro puntos: una impresora FDM no fabrica cualquier geometría igual de bien.

- **Voladizos:** una superficie que sobresale en el aire sin apoyo necesita soporte, o se cae mientras se imprime. Si puedes evitarlo reorientando la pieza, hazlo.
- **Orientación:** la pieza es más fuerte a lo largo de las capas que entre ellas. Orienta el chasis para que las fuerzas del uso normal —el peso de los motores, el impacto del balón contra las paletas— no tiendan a separar capas.
- **Anisotropía:** por lo anterior, una pieza impresa en 3D nunca es igual de resistente en todas direcciones. No es un defecto de tu impresión: es una propiedad del proceso.
- **Agujeros y tornillos:** un agujero impreso sale ligeramente más angosto que su medida nominal. Diseña los agujeros para tornillos con 0.2–0.3 mm de holgura extra.

3.4 Laminando en Cura

Laminar (*slicing*) es convertir tu modelo 3D en instrucciones capa por capa que la impresora entiende: el código G.

1. Abre Cura, elige el perfil de tu impresora e importa el archivo `chasis-robot.stl` que exportaste en el Capítulo 2. Si el modelo aparece con rayas grises, se sale del volumen de impresión: vuelve a OnShape y redúcelo.

2. Selecciona el perfil de material PLA y ajusta la altura de capa (0.2 mm es un buen punto de partida: buen acabado sin imprimir demasiado lento).
3. Define el relleno (20 % alcanza para el chasis) y activa soportes solo en los voladizos que identificaste en la sección anterior.
4. Verifica la adherencia a la cama: un borde (*brim*) ayuda si la pieza tiene poca superficie de contacto con la base.
5. Exporta el código G a la tarjeta SD o envíalo directo a la impresora.

La Figura 3.2 muestra el modelo importado, los ajustes y la vista previa por capas.



(**a**) Modelo importado (**b**) Ajustes de impresión (**c**) Vista previa por capas

Figura 3.2. Preparación del chasis en Ultimaker Cura.

CONSEJO PRÁCTICO

La vista previa por capas de Cura simula la impresión completa antes de gastar filamento. Revísala siempre: si ves soportes flotando en el aire o una base de contacto muy pequeña, corrige la orientación antes de imprimir, no después.

3.5 Manos a la obra: imprime tu chasis

MANOS A LA OBRA

1. Carga el código G de tu chasis en la impresora.
2. Nivela la cama si la impresora lo requiere, siguiendo su manual.
3. Inicia la impresión y supervisa las primeras capas: son las que definen si la pieza se pega bien a la base.
4. Cuando termine, retira la pieza con cuidado y quita los soportes con un alicate o una espátula.

3.6 Fallas típicas

ANTES DE SEGUIR, VERIFICA

- ☐ Identificaste los voladizos de tu chasis y decidiste su orientación.
- ☐ El chasis cabe en el volumen de impresión: en Cura no aparece con rayas grises.
- ☐ Laminaste el modelo en Cura con altura de capa 0.2 mm y relleno 20 %.
- ☐ Revisaste la vista previa por capas antes de imprimir.

Tabla 3.2. Diagnóstico rápido de fallas comunes en impresión FDM.

Síntoma	Causa probable	Solución
Las esquinas se levantan de la cama (*warping*)	Mala adherencia o enfriamiento desigual	Usa *brim*, limpia la cama, evita corrientes de aire
Hilos finos entre piezas (*stringing*)	Temperatura muy alta o falta de retracción	Baja 5–10 °C la boquilla, activa retracción
La pieza se despega a mitad de impresión	Cama sucia, sin nivelar, o primera capa muy rápida	Limpia con alcohol isopropílico, nivela, reduce velocidad de la primera capa

☐ Tu chasis está impreso, sin soportes, y encaja con las medidas que tomaste en el Capítulo 2.

RESUMEN DEL CAPÍTULO

- La manufactura aditiva construye piezas capa por capa; la FDM lo hace fundiendo filamento.
- El PLA es el material correcto para empezar: fácil de imprimir y suficiente para este chasis.
- Orientación, voladizos y anisotropía son decisiones de diseño, no detalles de la impresora.
- La mayoría de las fallas de impresión se resuelven en la primera capa o en la temperatura, no cambiando de máquina.

4 Electrónica

EN ESTE CAPÍTULO APRENDERÁS

- Entender qué hace cada componente eléctrico y electrónico del robot.
- Saber por qué un motor DC nunca se conecta directo al NodeMCU.
- Armar la alimentación del robot sin quemar ningún componente.
- Cablear y montar todos los componentes sobre tu chasis, en etapas que puedes verificar una por una.

Este es el capítulo donde más fácil es dañar un componente. No es motivo para tener miedo, sino para ir por etapas: cada sección de armado más adelante trae su propia verificación, y no vas a energizar nada que no puedas revisar antes.

4.1 Conceptos base

Tres cantidades explican casi todo lo que vas a hacer en este capítulo. El **voltaje** es la presión eléctrica disponible, en voltios (V). La **corriente** es cuánta electricidad realmente fluye, en amperios (A) — un motor detenido a la fuerza pide más corriente, no más voltaje. La **resistencia**, en ohmios (Ω), es lo que se opone a ese flujo. Y **GND** (tierra o masa) es el punto de referencia común: todo voltaje se mide *respecto a* GND, así que todos los GND del robot deben estar conectados entre sí.

4.2 Descripción de componentes

NOTA

El shield `HW-588A` trae su propio controlador de motores (un `L293D`), con borneras marcadas `A` y `B`. En este robot no se usa: entrega apenas 0.6 A por canal, y el `L298N` soporta bastante más. Las borneras `A`, `B` y `VM` del shield quedan libres; lo que sí usas son sus pines y su bornera de alimentación `VIN`.

4.3 Por qué necesitas un puente H

ATENCIÓN

Nunca conectes un motor DC directamente a un pin del NodeMCU. El pin entrega apenas unos 12 mA a 3.3 V, y el motor demanda cientos de miliamperios en el arranque. Vas a

Tabla 4.1. Los componentes centrales de este capítulo.

Componente	Qué hace	Dato clave
Motor DC + reductora	Convierte electricidad en giro; la reductora baja las RPM y multiplica el torque	Pide cientos de mA al arrancar
Servomotor (SG90/MG90S)	Gira hasta un ángulo específico según la señal que recibe; no da vueltas completas	Señal PWM a 50 Hz, pulso de 0.5 a 2.4 ms
Módulo `L298N`	Puente H: controla sentido y velocidad de motores DC que piden más corriente de la que entrega el microcontrolador	Hasta ~2 A por canal; cae ~2 V
NodeMCU ESP8266	Microcontrolador: ejecuta el programa y genera las señales de control. Trae WiFi, que este robot no usa	Trabaja a 3.3 V; cada pin entrega solo ~12 mA
Shield `HW-588A`	Base donde se enchufa el NodeMCU; reparte la alimentación y da pines para	Recibe entre 4.5 y 9 V por su bornera `VIN`

La velocidad se controla con **PWM** (Modulación por Ancho de Pulso): en vez de entregar un voltaje constante, el pin enciende y apaga muchas veces por segundo. Cuanto más tiempo pasa encendido en cada ciclo, más rápido gira el motor. `ENA` y `ENB` son los pines del `L298N` que reciben esa señal PWM, uno por motor.

quemar el pin, y en el peor caso, el microcontrolador completo.

El módulo `L298N` resuelve esto: el NodeMCU le manda una señal de control de apenas unos

NOTA

El NodeMCU trabaja a 3.3 V y el `L298N` a 5 V. No hace falta adaptar nada: el `L298N` reconoce como `HIGH` cualquier señal sobre unos 2.3 V, así que los 3.3 V del NodeMCU le bastan. Lo mismo vale para los servos y para el `HC-05`.

Tabla 4.2. Conexión entre el NodeMCU (pines del shield) y el módulo L298N.

NodeMCU	L298N	Función
`D8`	`ENA`	Velocidad del motor izquierdo (PWM)
`D7`	`IN1`	Sentido del motor izquierdo
`D6`	`IN2`	Sentido del motor izquierdo
`D3`	`ENB`	Velocidad del motor derecho (PWM)
`D5`	`IN3`	Sentido del motor derecho
`D4`	`IN4`	Sentido del motor derecho
`GND`	`GND`	Masa común, obligatoria

ATENCIÓN

El `L298N` viene de fábrica con un jumper sobre `ENA` y otro sobre `ENB`, que dejan los motores siempre a velocidad máxima. **Retíralos** antes de conectar los cables de `D8` y `D3`: si los dejas, el NodeMCU no puede regular la velocidad.

NOTA

`D3`, `D4` y `D8` cumplen además un papel al encender el ESP8266: de su nivel depende que arranque tu programa o que espere uno nuevo. Con el `L298N` funcionan sin problema, pero si algún día el NodeMCU no arranca con todo conectado, desconecta esos tres cables y prueba de nuevo (Anexo B).

4.4 Servomotores y ancho de pulso

Un servomotor también se controla por ancho de pulso, pero con otro significado: el pulso —repetido 50 veces por segundo— le dice al servo *a qué ángulo* moverse, no qué tan rápido girar. Un pulso de 0.5 ms lo manda a un extremo, uno de 2.4 ms al otro, y uno de unos 1.5 ms al centro.

Tus dos servos van conectados a `D0` (paleta izquierda) y `D1` (paleta derecha). En el ESP8266 casi cualquier pin puede generar esa señal.

NOTA

En el shield, el pin `D1` también llega a su controlador de motores interno. Como sus borneras `A` y `B` no tienen nada conectado, la señal del servo no tiene ningún efecto ahí.

ATENCIÓN

Cada paleta necesita un **tope mecánico** y un **rango angular acotado por software**. Un servo SG90 fuerza su engranaje plástico —y lo rompe— si el brazo choca contra el chasis antes de terminar su recorrido. En el Capítulo 5 vas a limitar el ángulo de cada servo en el código; por ahora, al montar la paleta, verifica a mano que se mueve libre en todo su rango antes de energizarla.

4.5 Alimentación

Tu robot se alimenta con **4 pilas AA en serie**, que entregan ~6 V, en un portapilas con interruptor. Toda esa energía entra primero al `L298N`, y desde ahí se reparte:

1. Las pilas van a la bornera de entrada del `L298N` (marcada `12V`) y a su `GND`. Con eso el módulo mueve los motores.
2. Con su jumper de 5 V puesto, el `L298N` enciende un regulador interno que entrega una tensión estable en su bornera `5V`.
3. Esa salida va a la bornera `VIN` del shield, junto con `GND`. Desde `VIN` se alimentan el NodeMCU —que la baja a 3.3 V con su propio regulador—, los dos servos y el módulo `HC-05`.

NOTA

El `L298N` pierde ~2 V al controlar los motores: con 6 V de entrada, los motores reciben ~4 V, no los 6 V nominales. Es normal, y conviene saberlo para no esperar más torque o velocidad de la que el sistema realmente entrega.

NOTA

El regulador del `L298N` necesita unos 7 V de entrada para entregar 5 V exactos. Con 4 pilas entrega algo menos, entre 4 y 5 V: alcanza para el NodeMCU, los servos y el `HC-05`, pero baja a medida que se gastan las pilas. Por eso este robot usa **pilas alcalinas nuevas**. Cuatro pilas recargables NiMH entregan solo 4.8 V en total, y con ellas el NodeMCU no recibe lo que necesita.

ATENCIÓN

En este montaje, el NodeMCU y los dos servos comparten la misma alimentación (`VIN` del shield, que viene de la salida de 5 V del `L298N`). Cuando las paletas se mueven, piden un pico de corriente que puede bajar esa tensión y reiniciar el NodeMCU. Instala un capacitor electrolítico de al menos 470 µF (16 V o más) entre `VIN` y `GND` del shield, cerca de los servos y con la polaridad correcta. **El capacitor no aumenta la capacidad del regulador.** Mide la tensión en `VIN` mientras se mueven ambas paletas; si el NodeMCU se reinicia, cambia las pilas, y si sigue pasando, alimenta los servos con una fuente aparte de 5 V, manteniendo GND común.

ATENCIÓN

GND común obligatorio. Las pilas, el `L298N`, el shield con el NodeMCU, los servos y el `HC-05` deben compartir la misma referencia de tierra. Sin GND común, las señales de control pierden su punto de referencia y el robot se comporta de forma errática o no responde.

CONSEJO PRÁCTICO

Las pilas AA alcalinas pierden voltaje rápido bajo el consumo de dos motores y dos servos, y ese voltaje caído se nota como pérdida de fuerza a medida que avanza la sesión de pruebas. Si notas que el robot rinde menos después de media hora, o que el NodeMCU se reinicia al patear, no es tu cableado: son las pilas. Ten siempre un juego nuevo a mano.

4.6 Manos a la obra: cableado y ensamble

Arma el robot en las siguientes etapas, en orden. Cada una trae su propia verificación: no avances a la siguiente sin pasarla. La Figura 4.1 muestra el esquema completo del cableado; consúltala en cada etapa. El módulo `HC-05` se conecta en el Capítulo 6.

MANOS A LA OBRA

1. **Portapilas al L298N.** Con el interruptor apagado, conecta el cable rojo del portapilas a la bornera `12V` del `L298N` y el negro a su `GND`. *Prueba:* enciende el interruptor; el LED del módulo se ilumina.
2. **Salida regulada del L298N.** Confirma que el jumper de 5 V está puesto. *Prueba:* con el

multímetro, mide entre 4 y 5 V entre la bornera `5V` del módulo y `GND`.

3. **Shield y NodeMCU.** Apaga el interruptor. Enchufa el NodeMCU en el shield, con la antena y el conector USB hacia donde indica la serigrafía. Lleva un cable desde la bornera `5V` del `L298N` a la bornera `VIN` del shield, y otro de `GND` a `GND`. *Prueba:* al encender, el LED azul del NodeMCU parpadea un instante, y entre `VIN` y `GND` del shield mides lo mismo que en el paso anterior.
4. **Motores al L298N.** Conecta el motor izquierdo a `OUT1/OUT2` y el derecho a `OUT3/OUT4`. *Prueba:* con el multímetro en modo continuidad, confirma que no hay corto entre las dos salidas de un mismo motor.
5. **Señales de control.** Retira los jumpers de `ENA` y `ENB` del `L298N` y cablea, con jumpers hembra-hembra, `ENA`, `IN1`–`IN4` y `ENB` según la Tabla 4.2. *Prueba:* revisa cable por cable contra la tabla; un pin cruzado aquí es el error más común de todo el capítulo.
6. **Servos al VIN del shield.** Instala el capacitor entre `VIN` y `GND` del shield, con su polo positivo a `VIN`. Conecta el cable rojo de cada servo a `VIN` —nunca a un pin de 3.3 V (`3V`)—, el café o negro a `GND` y la señal (naranjo o amarillo) a `D0` el izquierdo y a `D1` el derecho. *Prueba:* verifica cada cable contra la Figura 4.1; cuando cargues la rutina del Capítulo 5, mide `VIN` mientras ambas paletas se mueven.
7. **Verificación de GND común.** Con todo cableado y el interruptor apagado, mide continuidad entre el `GND` del `L298N`, el `GND` del shield y el `GND` de los servos. *Prueba:* el multímetro debe marcar continuidad (~0 Ω) entre los tres puntos.
8. **Montaje sobre el chasis.** Fija cada componente en su lugar en el chasis impreso en el Capítulo 3, con el cableado ordenado, sin tensión en los conectores y con el USB del NodeMCU accesible.

ANTES DE SEGUIR, VERIFICA

- [ ] El LED del módulo `L298N` enciende al conectar las pilas.
- [ ] La salida de 5 V del módulo mide entre 4 y 5 V con el multímetro.
- [ ] El NodeMCU enciende montado en el shield, alimentado por `VIN`.
- [ ] Los jumpers de `ENA` y `ENB` están retirados y cada pin de la Tabla 4.2 está cableado en el lugar correcto.
- [ ] Los servos toman energía de `VIN`, con el capacitor de desacople instalado, y sus señales llegan a `D0` y `D1`.
- [ ] Hay continuidad de GND entre `L298N`, shield y servos.
- [ ] Todos los componentes están fijos sobre el chasis.

RESUMEN DEL CAPÍTULO

- Un motor DC nunca se conecta directo al NodeMCU: el `L298N` existe para manejar esa corriente.
- PWM controla velocidad en los motores y ángulo en los servos, con el mismo principio de pulsos y distinto significado.
- Las 4 pilas alimentan el `L298N`; su salida regulada llega al `VIN` del shield y de ahí al NodeMCU, los servos y el `HC-05`. Hay que verificar esa tensión bajo carga y usar pilas en buen estado.

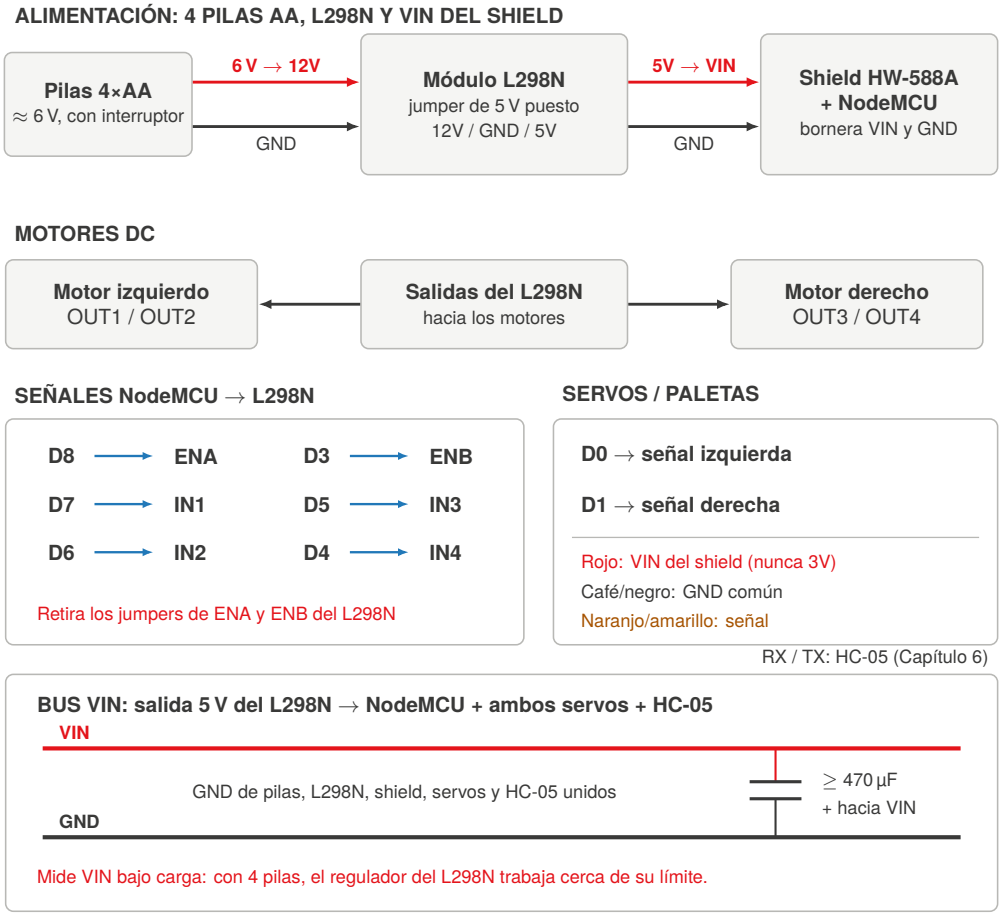


Figura 4.1. Esquema de conexión completo: pilas, L298N, shield con el NodeMCU, motores y servos.

- GND común no es opcional: sin él, el robot se comporta de forma errática aunque todo lo demás esté bien cableado.
- El armado en etapas verificables evita que un error de cableado aparezca recién cuando todo está montado y es difícil de revisar.

5 Programación

EN ESTE CAPÍTULO APRENDERÁS

- Entender qué es programar y por qué el NodeMCU se programa en C++.
- Preparar el Arduino IDE para programar el ESP8266.
- Reconocer la estructura de un programa: `setup()` y `loop()`.
- Escribir funciones que muevan un motor, luego dos, y finalmente ambos servos.
- Cargar una rutina de prueba completa y verificar que todo el robot responde.

5.1 ¿Qué es programar?

Programar es escribir instrucciones que una máquina ejecuta en orden, una tras otra. Un lenguaje de programación es *de bajo nivel* cuando esas instrucciones están cerca de lo que el procesador entiende directamente, y *de alto nivel* cuando están más cerca de cómo piensa una persona.

Tu NodeMCU se programa en **C++**, un lenguaje de nivel medio: te deja controlar cada pin directamente, pero con una sintaxis más cómoda que el ensamblador. El programa corre solo sobre el chip, sin sistema operativo de por medio, y C++ te da ese control fino sin desperdiciar memoria ni tiempo. Aunque el NodeMCU no es un Arduino, usa las mismas instrucciones: todo lo que aprendas aquí sirve también para una placa Arduino.

5.2 Prepara el Arduino IDE para el ESP8266

El Arduino IDE no trae de fábrica las herramientas para el ESP8266: se agregan una sola vez, como un paquete.

MANOS A LA OBRA

1. En el Arduino IDE abre *Archivo* → *Preferencias* y, en «Gestor de URLs adicionales de tarjetas», pega esta dirección: `https://arduino.esp8266.com/stable/package_esp8266com_index.json`
2. Abre *Herramientas* → *Placa* → *Gestor de tarjetas*, busca `esp8266` e instala el paquete «esp8266 by ESP8266 Community».
3. En *Herramientas* → *Placa*, elige **NodeMCU 1.0 (ESP-12E Module)**.
4. Conecta el NodeMCU por USB y elige su puerto en *Herramientas* → *Puerto*. Si no aparece ninguno, instala el controlador `CP2102` de Silicon Labs y prueba con otro cable: muchos solo cargan, no transmiten datos.

NOTA

Mientras programas, deja apagado el interruptor de las pilas: el USB alimenta el NodeMCU por sí solo. Enciende las pilas solo cuando el programa necesite mover los motores.

5.3 Estructura de un programa

Todo programa del Arduino IDE tiene dos funciones obligatorias:

- `setup()` se ejecuta una sola vez, al encender o reiniciar. Aquí configuras los pines.
- `loop()` se repite sin parar mientras la placa tenga corriente. Aquí vive el comportamiento del robot.

El Código 5.1 usa las tres instrucciones más básicas: `pinMode` declara si un pin es entrada o salida, `digitalWrite` lo pone en `HIGH` o `LOW`, y una variable (`const int LED`) le da nombre a un número de pin para no repetirlo por todo el código.

```
// Enciende y apaga el LED integrado del NodeMCU cada segundo.
// En el ESP8266 este LED funciona al revés: LOW lo enciende, HIGH lo apaga.
const int LED = LED_BUILTIN;

void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  digitalWrite(LED, LOW);   // encendido
  delay(1000);
  digitalWrite(LED, HIGH);  // apagado
  delay(1000);
}
```

Código 5.1. Parpadeo del LED integrado del NodeMCU.

NOTA

El LED integrado del NodeMCU está conectado «al revés»: se enciende con `LOW` y se apaga con `HIGH`. No es un error del programa; es cómo está cableado en la placa. Ese mismo pin es `D4`, que en tu robot también va al `L298N`: haz esta prueba con el interruptor de las pilas apagado.

NOTA

`analogWrite` funciona distinto de `digitalWrite`: en vez de `HIGH/LOW`, recibe un número y genera una señal PWM. Es la instrucción que vas a usar para controlar la velocidad de los motores. En el ESP8266, la escala de ese número cambió entre versiones del paquete (0 a 255 en las nuevas, 0 a 1023 en las antiguas); por eso los programas del libro llaman a `analogWriteRange(255)` en `setup()` y trabajan siempre de 0 a 255.

5.4 De un motor a dos: funciones y tracción diferencial

Una **función** le pone nombre a un bloque de instrucciones que vas a repetir. Sin funciones, mover dos motores en cuatro combinaciones de sentido —avanzar, retroceder, girar a cada lado— te obliga a repetir el mismo patrón de `digitalWrite` una y otra vez, con más oportunidades de equivocarte en algún cable.

El Código 5.2 define una función `moverMotor()` que recibe qué pines usar, a qué velocidad y en qué sentido, y encima construye `avanzar()`, `retroceder()`, `girarIzquierda()` y `girarDerecha()` llamando a esa función una vez por motor. Los pines se nombran como están impresos en la placa (`D8`, `D7`…): el paquete del ESP8266 ya sabe a qué patita del chip corresponde cada uno.

```
// Control de los dos motores DC con el módulo L298N (ver Tabla 4.2, Capítulo 4).
const int ENA = D8, IN1 = D7, IN2 = D6;  // Motor izquierdo
const int ENB = D3, IN3 = D5, IN4 = D4;  // Motor derecho

// Mueve un motor hacia adelante o hacia atrás, con velocidad de 0 a 255.
// En el robot del libro, IN1 en LOW e IN2 en HIGH hacen avanzar la rueda.
void moverMotor(int in1, int in2, int en, int velocidad, bool adelante) {
  digitalWrite(in1, adelante ? LOW : HIGH);
  digitalWrite(in2, adelante ? HIGH : LOW);
  analogWrite(en, velocidad);
}

void avanzar(int velocidad) {
  moverMotor(IN1, IN2, ENA, velocidad, true);
  moverMotor(IN3, IN4, ENB, velocidad, true);
}

void retroceder(int velocidad) {
  moverMotor(IN1, IN2, ENA, velocidad, false);
  moverMotor(IN3, IN4, ENB, velocidad, false);
}

void girarIzquierda(int velocidad) {
  moverMotor(IN1, IN2, ENA, velocidad, false);
  moverMotor(IN3, IN4, ENB, velocidad, true);
}

void girarDerecha(int velocidad) {
  moverMotor(IN1, IN2, ENA, velocidad, true);
  moverMotor(IN3, IN4, ENB, velocidad, false);
}

void detener() {
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
}

void setup() {
  analogWriteRange(255);  // velocidades de 0 a 255, igual que en un Arduino
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(ENA, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  pinMode(ENB, OUTPUT);
}

```

```
48 void loop() {
49   avanzar(150);
50   delay(1000);
51   detener();
52   delay(500);
53 }
```

Código 5.2. Funciones de movimiento sobre el módulo L298N.

CONSEJO PRÁCTICO

Si tu robot avanza cuando debería retroceder, no cambies el código: lo más probable es que un motor esté cableado al revés. Intercambia sus dos cables en la bornera del `L298N` antes de tocar una sola línea de `moverMotor()`.

5.5 Servos y la función de pateo

La librería `Servo.h`, que el paquete del ESP8266 trae incluida, simplifica el control por ancho de pulso que viste en el Capítulo 4: en vez de calcular pulsos, llamas a `servo.write(ángulo)` con un número de 0 a 180. Al conectar cada servo con `attach()` se le indican los pulsos de sus extremos, 500 y 2400 microsegundos, para que esos 180 grados correspondan al recorrido completo de un SG90.

El Código 5.3 declara los dos servos, los deja en su ángulo de reposo, y define `patear()`: mueve la paleta al ángulo de pateo y la regresa al reposo. Como los servos van montados en espejo, uno a cada lado del chasis, para girar hacia el balón uno baja de 90° a 40° y el otro sube a 140°: ambos se alejan solo 50° del reposo, bien lejos del límite mecánico, siguiendo la advertencia del Capítulo 4 sobre no forzar el engranaje.

```
1  // Control de las dos paletas (servos) con rango angular acotado.
2  // El límite entre reposo y pateo evita forzar el engranaje (ver Capítulo 4).
3  #include <Servo.h>
4
5  Servo servoIzquierdo, servoDerecho;
6  const int PIN_SERVO_IZQ = D0, PIN_SERVO_DER = D1;
7  const int ANGULO_REPOSO = 90;
8  // Servos montados en espejo: para ir hacia el balón, uno baja y el otro sube.
9  const int PATEO_IZQ = 40, PATEO_DER = 140;
10
11 void patear(Servo &servo, int anguloPateo) {
12   servo.write(anguloPateo);
13   delay(200);
14   servo.write(ANGULO_REPOSO);
15 }
16
17 void setup() {
18   // Pulsos de 0.5 a 2.4 ms: el recorrido completo de un SG90.
19   servoIzquierdo.attach(PIN_SERVO_IZQ, 500, 2400);
20   servoDerecho.attach(PIN_SERVO_DER, 500, 2400);
21   servoIzquierdo.write(ANGULO_REPOSO);
22   servoDerecho.write(ANGULO_REPOSO);
23 }
24
25 void loop() {
26   patear(servoIzquierdo, PATEO_IZQ);
27   delay(1000);
```

```
  patear(servoDerecho, PATEO_DER);
  delay(1000);
}
```

Código 5.3. Control de las paletas con rango angular acotado.

CONSEJO PRÁCTICO

Si una paleta gira hacia afuera en vez de ir hacia el balón, cambia su ángulo de pateo por su simétrico: 180 menos el valor actual (40 pasa a 140, y al revés).

5.6 Manos a la obra: tu rutina de prueba

MANOS A LA OBRA

1. Descarga del repositorio del libro (Anexo C) el archivo `codigo/rutina-prueba/rutina-prueba.ino`, que integra las funciones de las dos secciones anteriores en un solo programa, y ábrelo en el Arduino IDE.
2. Conecta el NodeMCU a tu computadora por USB y súbelo con el botón de carga.
3. Con el robot sobre una superficie donde las ruedas giren libres, enciende el interruptor de las pilas.
4. Observa el ciclo completo: avanza, retrocede, gira a la izquierda, gira a la derecha, patea con el servo izquierdo y patea con el derecho.
5. Mientras se mueven las paletas, mide la tensión entre `VIN` y `GND` del shield, como pedía el Capítulo 4.

ATENCIÓN

Si alguna función no responde —un motor no gira, un servo no se mueve— **no insistas con el código**. Vuelve al Capítulo 4 y revisa esa etapa específica del armado: en la enorme mayoría de los casos, un motor o servo mudo es un cable en el pin equivocado o una alimentación mal conectada, no un error de programación.

5.7 Depuración con el Monitor Serie

La rutina de prueba llama a `Serial.begin(9600)` en `setup()`, y a `Serial.println()` antes de cada movimiento. Abre el Monitor Serie del Arduino IDE, a 9600 baudios y con el NodeMCU conectado por USB, para ver en texto qué función se está ejecutando en cada momento.

NOTA

Justo al encender, el ESP8266 envía por el puerto serial un mensaje de arranque a otra velocidad, que en el Monitor Serie aparece como una línea de caracteres extraños. Es normal: tu programa empieza a escribir después.

CONSEJO PRÁCTICO

El Monitor Serie es la herramienta más simple para saber si tu programa llegó a una parte del código: si el mensaje «Girar izquierda» nunca aparece, el problema está antes de esa línea, no después.

ANTES DE SEGUIR, VERIFICA

- [ ] El Arduino IDE tiene instalado el paquete del ESP8266 y reconoce el puerto del NodeMCU.
- [ ] El LED integrado del NodeMCU parpadea al cargar el Código 5.1.
- [ ] El robot avanza y retrocede en línea recta al ejecutar `avanzar()` y `retroceder()`.
- [ ] El robot gira hacia cada lado al ejecutar `girarIzquierda()` y `girarDerecha()`.
- [ ] Ambos servos patean hacia el balón y vuelven a su posición de reposo sin forzar el engranaje.
- [ ] El Monitor Serie muestra el nombre de cada movimiento mientras el robot lo ejecuta.

RESUMEN DEL CAPÍTULO

- El NodeMCU se programa desde el Arduino IDE, en C++, después de instalar una vez el paquete del ESP8266.
- Todo programa tiene `setup()` (una vez) y `loop()` (siempre).
- Las funciones evitan repetir código y hacen visibles los errores de cableado: si `avanzar()` falla, sabes exactamente qué revisar.
- La tracción diferencial se logra combinando el sentido de giro de los dos motores, no con hardware adicional.
- El rango angular de los servos se acota en el código para proteger el engranaje físico.
- El Monitor Serie es tu primera herramienta de diagnóstico, antes de sospechar del cableado.

6 Comunicación

EN ESTE CAPÍTULO APRENDERÁS

- Entender qué es un protocolo de comunicación y dónde encaja Bluetooth.
- Conectar el módulo HC-05 al NodeMCU sin dañarlo.
- Conocer el protocolo de comandos de tu robot y programarlo para recibirlos.
- Instalar la app de control hecha en MIT App Inventor, entender cómo está construida y adaptarla.

6.1 ¿Qué es un protocolo, y por qué Bluetooth?

Un **protocolo** es un acuerdo: dos dispositivos deciden de antemano qué significa cada dato que se envían, para poder entenderse. Un cable USB y una red WiFi son ambos «alámbrico» y «inalámbrico», pero por debajo usan protocolos distintos: `UART`, `I2C` y `SPI` para conectar chips dentro de un mismo circuito; WiFi y **Bluetooth** para comunicar dispositivos separados sin cables.

Tu NodeMCU trae WiFi integrado, pero para controlar el robot desde una app de MIT App Inventor el camino más directo es el Bluetooth: todo celular lo trae, y el módulo que lo resuelve —el HC-05— es barato y se conecta con cuatro cables.

6.2 El módulo HC-05 y el puerto serial

El **HC-05** es un módulo Bluetooth que convierte lo que envías por comunicación serial en una señal Bluetooth, y viceversa. Desde el punto de vista del NodeMCU, hablar con el HC-05 es igual de simple que hablar con el Monitor Serie del Capítulo 5: de hecho, usa el mismo puerto.

En este robot el HC-05 va conectado a los pines seriales de hardware del NodeMCU, `RX` y `TX`. Es la conexión más simple: no necesita ninguna librería, y el programa lee los comandos con el mismo `Serial` que ya usaste para depurar.

ATENCIÓN

Los pines `RX` y `TX` son los mismos que usa el USB para cargar el programa. **Desconecta el HC-05 —o al menos su cable `TXD`— cada vez que subas un programa nuevo**, y vuelve a conectarlo después. Si lo dejas, la carga falla con un error de conexión, aunque el código esté perfecto.

Tabla 6.1. Conexión entre el NodeMCU (pines del shield) y el módulo HC-05.

NodeMCU	HC-05	Función
RX	TXD	El HC-05 transmite, el NodeMCU recibe
TX	RXD	El NodeMCU transmite, el HC-05 recibe
VIN	VCC	Alimentación del módulo
GND	GND	Masa común, obligatoria

NOTA

Las líneas de datos van **cruzadas**: lo que uno transmite (TX) es lo que el otro recibe (RX). Conectar TX con TXD es el error más común de esta etapa, y no daña nada, pero tampoco funciona. Los pines EN y STATE del HC-05 quedan sin conectar.

NOTA

Con una placa Arduino de 5 V haría falta un divisor de tensión antes de la entrada RXD del módulo, que espera 3.3 V. Con el NodeMCU no: trabaja a 3.3 V, igual que el HC-05, y los dos se conectan directo. La alimentación VCC sí va a VIN, porque el módulo acepta entre 3.6 y 6 V.

6.3 El protocolo de comandos del robot

El protocolo lo define la app de control: cada botón envía una sola letra, y el NodeMCU hace algo distinto con cada una. Los botones de dirección envían su letra al presionarlos y una S al soltarlos, así que el robot se mueve solo mientras mantienes el dedo encima.

Tabla 6.2. Protocolo de comandos del robot (app App_Robotiza_2).

Carácter	Acción	Carácter	Acción
U	Avanzar	Q	Avanzar curvando a la izquierda
D	Retroceder	E	Avanzar curvando a la derecha
L	Girar a la izquierda	Z	Retroceder curvando a la izquierda
R	Girar a la derecha	C	Retroceder curvando a la derecha
S	Detenerse	A	Patear con la paleta izquierda
		B	Patear con la paleta derecha

El Código 6.1 retoma las funciones del Capítulo 5 y agrega dos piezas. La primera es `mover()`, que fija la velocidad y el sentido de cada rueda por separado: es la tracción diferencial completa en una sola función. Las cuatro diagonales salen de ahí sin hardware extra: la rueda del lado hacia donde se curva gira más lento (`VELOCIDAD_CURVA`) que la otra, y el robot traza un arco. La segunda pieza es `ejecutarComando()`, que traduce cada letra de la Tabla 6.2 en un movimiento. En `loop()`, el NodeMCU solo pregunta si llegó un comando nuevo —no hace falta nada más.

```
// Control del robot por Bluetooth con la app App_Robotiza_2 (Capítulo 6).
// El HC-05 va al puerto serial del NodeMCU (RX y TX): desconéctalo antes de
// subir el programa. Un carácter por comando (ver Tabla 6.2).
#include <Servo.h>

// Motores (módulo L298N, ver Tabla 4.2, Capítulo 4)
```

```
const int ENA = D8, IN1 = D7, IN2 = D6;  // Motor izquierdo
const int ENB = D3, IN3 = D5, IN4 = D4;  // Motor derecho
const int VELOCIDAD = 200;        // rectas y giros en el lugar
const int VELOCIDAD_CURVA = 90;   // rueda interior en las diagonales

// Servos: paletas tipo pinball (Capítulo 4)
Servo servoIzquierdo, servoDerecho;
const int PIN_SERVO_IZQ = D0, PIN_SERVO_DER = D1;
const int ANGULO_REPOSO = 90;
const int PATEO_IZQ = 40, PATEO_DER = 140;

void moverMotor(int in1, int in2, int en, int velocidad, bool adelante) {
  digitalWrite(in1, adelante ? LOW : HIGH);
  digitalWrite(in2, adelante ? HIGH : LOW);
  analogWrite(en, velocidad);
}

// Velocidad y sentido de cada rueda: con esto se arma cualquier movimiento.
void mover(int velIzq, bool adelanteIzq, int velDer, bool adelanteDer) {
  moverMotor(IN1, IN2, ENA, velIzq, adelanteIzq);
  moverMotor(IN3, IN4, ENB, velDer, adelanteDer);
}

void detener() {
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
}

void patear(Servo &servo, int anguloPateo) {
  servo.write(anguloPateo);
  delay(200);
  servo.write(ANGULO_REPOSO);
}

void ejecutarComando(char comando) {
  const int V = VELOCIDAD, VC = VELOCIDAD_CURVA;  // nombres cortos
  switch (comando) {
    case 'U': mover(V, true,  V, true);  break;  // adelante
    case 'D': mover(V, false, V, false); break;  // atrás
    case 'L': mover(V, false, V, true);  break;  // gira a la izquierda
    case 'R': mover(V, true,  V, false); break;  // gira a la derecha
    case 'Q': mover(VC, true,  V, true);  break;  // adelante a la izquierda
    case 'E': mover(V, true,  VC, true);  break;  // adelante a la derecha
    case 'Z': mover(VC, false, V, false); break;  // atrás a la izquierda
    case 'C': mover(V, false, VC, false); break;  // atrás a la derecha
    case 'S': detener(); break;
    case 'A': patear(servoIzquierdo, PATEO_IZQ); break;
    case 'B': patear(servoDerecho, PATEO_DER); break;
    // 'I' y 'P' (seguidor de línea) o cualquier otro carácter: se ignoran.
  }
}

void setup() {
  Serial.begin(9600);  // velocidad de fábrica del HC-05
  analogWriteRange(255);

  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(ENA, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  pinMode(ENB, OUTPUT);
  detener();
```

```
70
71    servoIzquierdo.attach(PIN_SERVO_IZQ, 500, 2400);
72    servoDerecho.attach(PIN_SERVO_DER, 500, 2400);
73    servoIzquierdo.write(ANGULO_REPOSO);
74    servoDerecho.write(ANGULO_REPOSO);
75  }
76
77  void loop() {
78    if (Serial.available()) {
79      ejecutarComando(Serial.read());
80    }
81  }
```

Código 6.1. Recepción de comandos por Bluetooth y control del robot.

NOTA

La app trae también un menú «Seguidor de líneas», que envía `I` y `P` para una versión del robot con sensores infrarrojos. Este libro no la cubre, y el programa simplemente ignora esas letras. Es un buen punto de partida para seguir después (Capítulo 7).

6.4 La app de control en MIT App Inventor

La app del robot, `App_Robotiza_2`, está en el repositorio del libro (Anexo C) en dos formatos: el proyecto `.aia`, que se abre y se edita en MIT App Inventor, y el instalador `.apk`, listo para el celular. Necesitas un celular **Android**: las apps de App Inventor con Bluetooth clásico no funcionan en iPhone.

MIT App Inventor tiene dos vistas: el **Diseñador**, donde se acomodan botones y otros elementos visuales, y **Bloques**, donde se define qué hace cada uno. Para abrir la app, entra a App Inventor y usa *Proyectos* → *Importar proyecto* (*.aia*) *desde mi ordenador*. Recórrela así:

1. **Las pantallas.** Todo vive en una sola pantalla con tres grupos (*arrangements*) que se muestran o se esconden: el menú principal, el control «RC Fútbol» y el «Seguidor de líneas». El componente `BluetoothClient` no es visible: es el que habla con el HC-05.
2. **La conexión.** El selector de lista «Conectar BT» (`ListPicker`) carga, antes de abrirse (`BeforePicking`), los dispositivos emparejados (`BluetoothClient1.AddressesAndNames`); al elegir uno (`AfterPicking`), llama a `BluetoothClient1.Connect` y avisa «Conectado!».
3. **La cruceta.** Los ocho botones de dirección usan dos eventos: `TouchDown` (al presionar) envía con `BluetoothClient1.SendText` la letra de esa dirección, según la Tabla 6.2, y `TouchUp` (al soltar) envía `S`. Sin este segundo bloque el robot no se detendría: el programa del NodeMCU solo frena al recibir `S`.
4. **Los botones A y B.** Basta el evento `Click`, que envía `A` o `B`: una patada por toque.
5. **La protección.** Antes de enviar, cada bloque revisa si hay conexión; si no la hay, muestra el aviso «Bluetooth no conectado» en vez de fallar en silencio.

La Figura 6.1 muestra el Diseñador, los Bloques y la app ya corriendo en el celular.



(**a**) Diseñador: cruceta y botones



(**b**) Bloques: enviar comando



(**c**) App corriendo en el celular

Figura 6.1. La app de control App_Robotiza_2 en MIT App Inventor.

CONSEJO PRÁCTICO

Para probar cambios en la app, usa la app «MIT AI2 Companion» en tu celular antes de exportar un `.apk` nuevo. Así detectas errores de bloques sin esperar a tener el robot encendido al lado.

6.5 Manos a la obra: empareja y controla tu robot

MANOS A LA OBRA

1. Con el HC-05 desconectado, sube el Código 6.1 al NodeMCU. Después conecta el HC-05 según la Tabla 6.1.
2. Enciende el robot: el LED del HC-05 parpadea rápido mientras espera conexión. Búscalo en el Bluetooth de tu celular como `HC-05` y empareja; el código por defecto suele ser `1234` o `0000`.
3. Instala la app: el `App_Robotiza_2.apk` del repositorio (el celular te pedirá permitir apps de origen desconocido), o el proyecto `.aia` a través del Companion.
4. Abre la app, toca «Conectar BT», elige el HC-05 y entra a «RC Fútbol». Al conectarse, el LED del módulo cambia a un parpadeo lento.
5. Prueba cada botón: las cuatro direcciones, las cuatro diagonales, y A y B. Mantén presionado un botón de dirección y suéltalo: el robot debe moverse mientras lo presionas y detenerse al soltarlo.
6. Si un botón no responde, revisa en Bloques la letra que envía contra la Tabla 6.2, y en el programa el `case` que la recibe: es el error más común en esta etapa.
7. **Hazla tuya.** Cambia `VELOCIDAD` y `VELOCIDAD_CURVA` en el programa hasta que el robot se maneje a tu gusto. Después, en App Inventor, agrega un botón que envíe una letra nueva —por ejemplo `K`, para patear con las dos paletas— y súmale su `case` al programa.

ANTES DE SEGUIR, VERIFICA

- ☐ El celular encuentra y empareja el HC-05.
- ☐ La app se conecta al módulo sin errores.
- ☐ Los botones de dirección y las diagonales mueven el robot mientras los presionas y lo detienen al soltarlos.
- ☐ Los botones A y B activan la paleta izquierda y la derecha.
- ☐ Sabes que el HC-05 se desconecta antes de cada carga de programa.

RESUMEN DEL CAPÍTULO

- Un protocolo es un acuerdo sobre qué significa cada dato: el de tu robot tiene once comandos, cada uno una sola letra.
- El HC-05 va al puerto serial de hardware del NodeMCU: se lee con `Serial`, sin librerías, pero hay que desconectarlo para cargar programas.
- NodeMCU y HC-05 trabajan a 3.3 V: se conectan directo, sin divisor de tensión, con las líneas de datos cruzadas.
- MIT App Inventor convierte cada botón en un mensaje Bluetooth de un carácter, que tu NodeMCU ya sabe interpretar; con el proyecto `.aia` puedes cambiar la app a tu gusto.

7 Epílogo

Felicidades. Si llegaste hasta aquí con el robot respondiendo a tu celular, no armaste solo un vehículo con motores: pasaste por las cuatro capas que el Capítulo 0 prometía —mecánica, eléctrica, electrónica e informática— y las hiciste convivir en una sola máquina.

ANTES DE SEGUIR, VERIFICA

- [ ] El chasis diseñado en OnShape está impreso y todos los componentes encajan en él.
- [ ] Los motores, el módulo L298N, el NodeMCU con su shield y los servos están cableados y alimentados sin GND flotante.
- [ ] El código de la rutina de prueba corrió sin errores en los seis movimientos.
- [ ] La app de MIT App Inventor controla el robot por Bluetooth: avanza, retrocede, gira, curva y patea.

7.1 Reglas para jugar

No necesitas más que una cancha improvisada y otro robot igual al tuyo. Marca un rectángulo de unos 2×1 m con cinta en el suelo, con un arco pequeño en cada extremo. Dos reglas alcanzan: gana quien mete el balón en el arco contrario, y ningún robot puede tocar al otro directamente —solo al balón.

7.2 Lo que aprendiste

Lo que realmente construiste es transferible: diseñar una pieza para que cumpla una función, entender por qué un motor necesita un puente H antes de recibir corriente, escribir funciones en vez de repetir código, y definir un protocolo simple para que dos dispositivos se entiendan. Esas cuatro habilidades sirven para el próximo robot, no solo para este.

7.3 Hacia dónde seguir

Con los mismos componentes —o agregando uno o dos sensores— puedes construir:

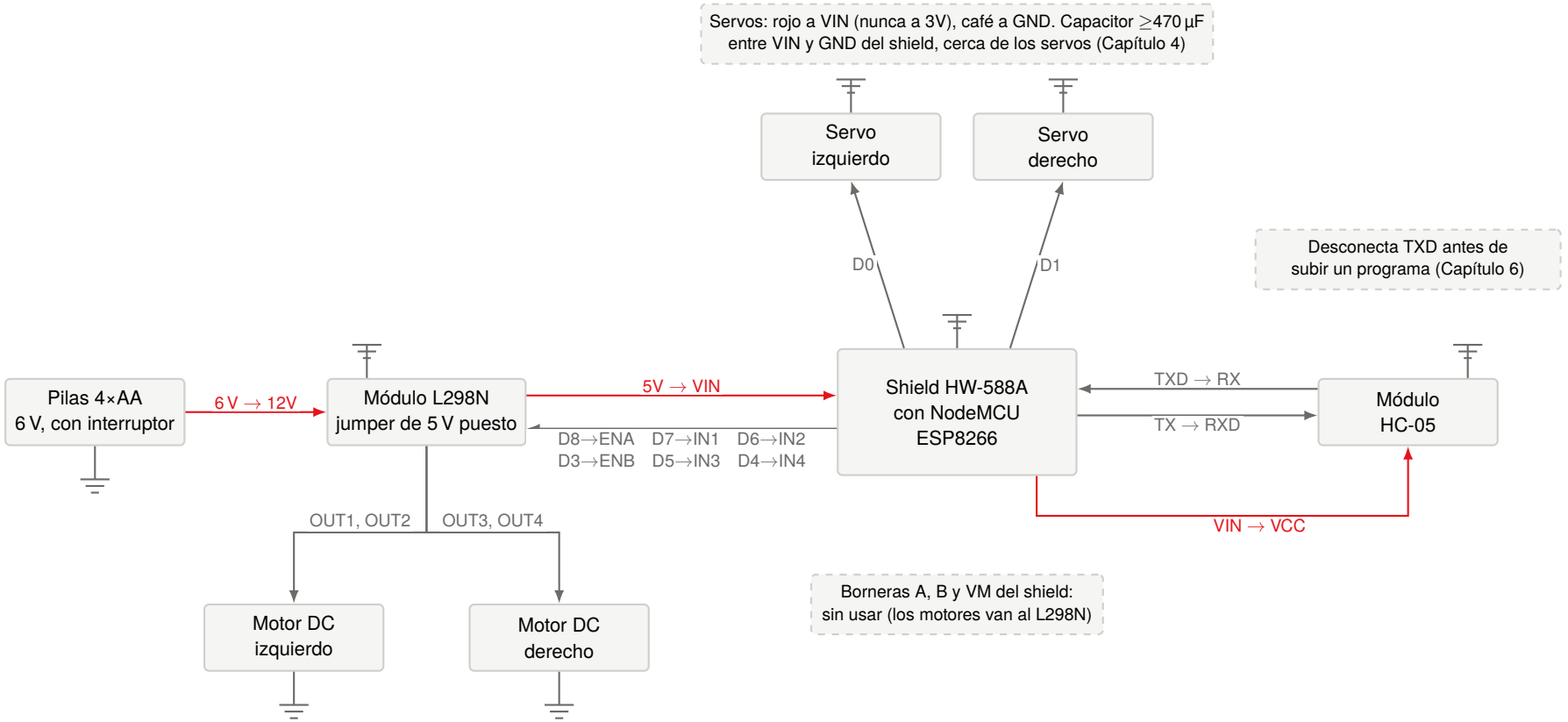
- Un **seguidor de línea**, agregando sensores infrarrojos: la app ya trae el menú y los comandos `I` y `P` para activarlo.
- Un **evasor de obstáculos**, con un sensor ultrasónico.
- Un **brazo robótico**, reutilizando lo que ya sabes de servos.

- Control por **WiFi** en vez de Bluetooth: tu NodeMCU ya lo trae integrado, sin ningún módulo extra.
- Un robot con **visión**, agregando una cámara y un poco de procesamiento de imágenes.

Cada uno de esos proyectos parte del mismo lugar donde tú ya estás parado.

Gracias por construir hasta el final. Ahora te toca jugar.

A Diagrama de conexiones



Todos los símbolos de tierra (⊥) están unidos entre sí: GND común obligatorio (Capítulo 4).

B Guía de resolución de problemas

Todas las fallas de esta guía ya aparecieron, una por una, en su capítulo correspondiente. Aquí están juntas, por síntoma, para revisar rápido sin tener que recorrer todo el libro.

Tabla B.1. Impresión 3D (Capítulo 3).

Síntoma	Causa probable	Solución
Las esquinas se levantan de la cama (*warping*)	Mala adherencia o enfriamiento desigual	Usa *brim*, limpia la cama, evita corrientes de aire
Hilos finos entre piezas (*stringing*)	Temperatura muy alta o falta de retracción	Baja 5–10 °C la boquilla, activa retracción
La pieza se despega a mitad de impresión	Cama sucia, sin nivelar, o primera capa muy rápida	Limpia con alcohol isopropílico, nivela, reduce velocidad de la primera capa

Tabla B.2. Electrónica y alimentación (Capítulo 4).

Síntoma	Causa probable	Solución
El NodeMCU se reinicia al mover un servo	La tensión de `VIN` cae: pilas gastadas, o el regulador del L298N no alcanza para el NodeMCU y los servos a la vez	Mide `VIN` con las paletas en movimiento. Cambia las pilas por alcalinas nuevas; si sigue pasando, alimenta los servos con una fuente de 5 V aparte (GND común) y conserva el capacitor de $\geq$470 µF
El NodeMCU no arranca, o su LED queda fijo, con todo conectado	`D3`, `D4` o `D8` quedan en un nivel que impide el arranque	Desconecta esos tres cables, enciende, y vuelve a conectarlos uno por uno para encontrar el culpable
El robot se comporta de forma errática o no responde	Falta GND común entre alguna etapa	Verifica continuidad de GND entre pilas, L298N, shield y servos
Los motores giran siempre a fondo, aunque cambies la velocidad	Siguen puestos los jumpers de `ENA` y `ENB` del L298N	Retíralos y conecta ahí los cables de `D8` y `D3`
Un motor gira al revés de lo esperado	Cables del motor invertidos en la bornera	Intercambia los dos cables de ese motor en el L298N
El robot pierde fuerza después de un rato de uso	Pilas alcalinas descargándose bajo carga	Cambia las pilas. Cuatro recargables NiMH entregan solo 4.8 V: no alcanzan para este montaje

Tabla B.3. Programación (Capítulo 5).

Síntoma	Causa probable	Solución
El Arduino IDE no muestra ningún puerto	Falta el controlador `CP2102`, o el cable USB solo carga	Instala el controlador de Silicon Labs y prueba con otro cable
La carga falla con un error de conexión	El HC-05 sigue conectado a `RX`/`TX`, o no está elegida la placa NodeMCU 1.0	Desconecta el HC-05 (al menos `TXD`) y revisa *Herramientas* → *Placa*
Aparecen caracteres extraños en el Monitor Serie al encender	Es el mensaje de arranque del ESP8266, enviado a otra velocidad	Es normal: ignóralo. Si todo lo que ves es ilegible, ajusta el Monitor Serie a 9600 baudios
Un motor o servo no responde al cargar el código	Cable en el pin equivocado, o alimentación desconectada	Vuelve al Capítulo 4 y revisa esa etapa específica antes de tocar el código
El robot avanza cuando debería retroceder	Motor cableado con polaridad invertida	Intercambia los dos cables de ese motor, no cambies el código
Una paleta gira hacia afuera en vez de hacia el balón	El servo está montado al revés de lo que supone el código	Cambia su ángulo de pateo por 180 menos el valor actual
Un servo fuerza o hace ruido al llegar a un extremo	Rango angular no acotado, choca contra el chasis	Ajusta `PATEO_IZQ`, `PATEO_DER` y `ANGULO_REPOSO` para que el brazo nunca llegue al tope mecánico

Tabla B.4. Comunicación (Capítulo 6).

Síntoma	Causa probable	Solución
El celular no encuentra el HC-05	Módulo sin alimentación, o fuera de rango	Verifica el LED del HC-05 (parpadeo rápido = listo para emparejar)
La app no logra conectar al módulo	Emparejamiento Bluetooth del celular no completado antes de abrir la app	Empareja primero desde los ajustes del celular, con el código `1234` o `0000`
La app no se instala o no encuentra el Bluetooth	Celular iPhone, o apps de origen desconocido bloqueadas	Usa un celular Android y permite instalar el `.apk`
La app dice «Bluetooth no conectado»	Se tocó un botón antes de conectar	Toca «Conectar BT» y elige el HC-05
La app conecta, pero el robot no reacciona	Líneas de datos sin cruzar (`TX` con `TXD`), o velocidad del módulo distinta de 9600	Conecta `TXD`→`RX` y `RXD`→`TX`. Si cambiaste la velocidad del HC-05 con comandos AT, usa esa misma en `Serial.begin()`
Un botón de la app no mueve el robot	La letra enviada en Bloques no coincide con el programa	Revisa el bloque `SendText` contra la Tabla 6.2 y el `case` correspondiente
El robot sigue moviéndose al soltar el botón	Falta el evento `TouchUp`: el NodeMCU solo frena al recibir `S`	En cada botón de dirección, agrega un bloque `TouchUp` que envíe `S`

C Archivos descargables

Todos los archivos del libro —el código para el NodeMCU, la app de control, el modelo 3D del chasis del autor y este mismo libro en PDF— están disponibles para descarga libre en el repositorio:

`https://github.com/Edward3th/UA_Robotica`

Los de esta guía están en la carpeta `guia-1-introduccion-a-la-robotica/`. Para bajar todo de una vez, usa el botón **Code** y luego **Download ZIP** en la página del repositorio; no necesitas una cuenta de GitHub.

Tabla C.1. Archivos de esta guía en el repositorio.

Carpeta o archivo	Contenido
`Introduccion-a-la-Robotica.pdf`	Este libro
`codigo/cap05-01-parpadeo/`	Parpadeo del LED integrado del NodeMCU (Capítulo 5)
`codigo/cap05-02-motores/`	Funciones de movimiento con el `L298N` (Capítulo 5)
`codigo/cap05-03-servos/`	Control de las paletas (Capítulo 5)
`codigo/rutina-prueba/`	Rutina de prueba de motores y servos (Capítulo 5)
`codigo/control-bluetooth/`	Programa de control por Bluetooth con el `HC-05` (Capítulo 6)
`app/App_Robotiza_2.aia`	Proyecto de la app, para abrir y editar en MIT App Inventor (Capítulo 6)
`app/App_Robotiza_2.apk`	App lista para instalar en un celular Android (Capítulo 6)
`chasis/chasis.stl`	Placa del chasis del autor (Capítulos 2 y 3)
`chasis/guardabarros.stl`	Guardabarros del chasis del autor
`chasis/union-motor-1.stl`	Pieza de unión de los motores al chasis
`chasis/union-motor-2.stl`	Segunda pieza de unión de los motores

NOTA

Cada programa está dentro de una carpeta con su mismo nombre, porque el Arduino IDE así lo exige. Abre el archivo `.ino` desde esa carpeta, sin moverlo ni renombrarlo, y súbelo con la placa **NodeMCU 1.0 (ESP-12E Module)** elegida (Capítulo 5).

NOTA

Si modificas la app en App Inventor, exporta de nuevo tanto el `.aia` (para seguir editando) como el `.apk` (para instalar).

ATENCIÓN

La placa del chasis del autor mide 250×130 mm y no cabe en las camas de 220×220 mm de muchas impresoras. Antes de laminarla, comprueba en Cura que quede dentro del

volumen de impresión. Si no cabe, úsala como referencia y diseña tu propia placa, como propone el Capítulo 2.

Bibliografía

[1] Arduino. Arduino Documentation. https://docs.arduino.cc/. Consultado el 2 de octubre de 2026.

[2] Arduino. Servo Library Reference. https://docs.arduino.cc/libraries/servo. Consultado el 2 de octubre de 2026.

[3] Arduino. SoftwareSerial Library Reference. https://docs.arduino.cc/learn/built-in-libraries/software-serial/. Consultado el 2 de octubre de 2026.

[4] Guangzhou HC Information Technology Co., Ltd. HC Serial Bluetooth Products: User Instructional Manual. https://components101.com/sites/default/files/component_datasheet/HC06%20Bluetooth%20Module%20Datasheet.pdf. Manual del fabricante, copia alojada por Components101. Consultado el 2 de octubre de 2026.

[5] MIT. MIT App Inventor Support. https://appinventor.mit.edu/explore/ai2/support/.

[6] Onshape. Onshape Help. https://cad.onshape.com/help/. Consultado el 2 de octubre de 2026.

[7] RoboCup Federation. RoboCup: sitio oficial. https://www.robocup.org/. Consultado el 2 de octubre de 2026.

[8] STMicroelectronics. L298 Dual Full-Bridge Driver. https://www.st.com/resource/en/datasheet/l298.pdf. Hoja de datos del fabricante.

[9] UltiMaker. UltiMaker Cura. https://ultimaker.com/software/ultimaker-cura/. Consultado el 2 de octubre de 2026.